

Editor: **Davide Taibi**
University of Oulu
davide.taibi@oulu.fi

Agentic AI Frameworks Under the Microscope

What Works, What Doesn't

Karthik Vaidhyanathan, IIIT Hyderabad

Davide Taibi, University of Southern Denmark and University of Oulu

From the Editor

AI agents are rapidly transforming the software engineering landscape, supported by a wave of new frameworks. Our review of the 10 most representative general-purpose agentic AI frameworks reveals a maturing but fragmented landscape. While some frameworks provide enterprise-ready capabilities, most remain immature, with limited memory models, basic reasoning, and uneven ecosystem support. Significant improvements are needed before these frameworks can enable reliable and large-scale adoption in practice.—*Davide Taibi*

Introduction

AI agents are becoming more common, and their growing use in software engineering is starting to shape developers' daily work. At the same time, a rapidly expanding set of tools and frameworks is emerging to support their development and adoption.

AI agents are autonomous entities that can sense their environment, make decisions, and accomplish tasks.[1] The combination of multiple AI agents working in concert has given rise to agentic AI systems, fundamentally shifting how we approach software development. This shift represents a change from fully deterministic preprogrammed workflows to more probabilistic adaptive systems that can handle uncertainty and evolving requirements. However, this promising landscape comes with a significant challenge emerging from the explosion of agentic AI frameworks.

This article presents the current landscape of agentic AI frameworks, which provide different capabilities and characteristics for developing and deploying agentic AI systems. This article focuses on agentic AI frameworks and does not cover low-code or no-code tools for building agents.

The Framework Selection Process

To systematically identify representative agentic AI frameworks, we conducted a structured search and selection process on GitHub, filtering repositories with more than 5,000 stars, yielding 243 candidates. We then applied inclusion criteria focusing on general-purpose frameworks that provide comprehensive

Digital Object Identifier 10.1109/MS.2025.3622209
Date of current version: 23 December 2025

This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/

capabilities for building agentic AI systems while excluding repositories that documented existing tools, collected research papers, or offered narrow single-purpose utilities.

To systematically evaluate the selected frameworks, we considered the following five key dimensions based on the existing literature on agentic architectures, observable characteristics in open source projects, and core principles of agentic system design[2,3]:

> Our review of the 10 most representative general-purpose agentic AI frameworks reveals a maturing but fragmented landscape.

1. *Goal handling and interaction:* This dimension assesses how an agentic AI framework interprets user intentions and manages interaction—whether it operates *passively*, relying on explicit prompts, or *proactively*, inferring goals from the environmental context. This is further broken down into three subcategories.
   - *Goal processing:* what the framework supports: passive agents that wait for explicit instructions (**Pa**) or proactive agents that can interpret context and infer next steps independently (**Pr**)
   - *Input modality:* what types of input the framework supports: text-only (**T**) interactions or multiple formats like voice, images, and documents (multimodal, **M**)
   - *Interaction pattern:* what communication styles the framework enables: one-time requests (single-shot, **S**), back-and-forth conversations (conversational, **C**), or agents that run continuously without human input (autonomous loops, **A**).
2. *Planning and reasoning:* Beyond goal interpretation, frameworks exhibit distinct approaches to task decomposition and execution strategy. Planning methodologies span from *sequential execution* following predetermined paths to multibranch exploration that evaluates alternative approaches to *reactive processing* that adapts dynamically without explicit forward planning. Reasoning mechanisms include *chain-of-thought deliberation*, *tool-first execution* that prioritizes action over reflection, and *hybrid approaches* that balance contemplation with action. This dimension is categorized by the following:
   - *Planning (**Pl**):* types of task breakdown the framework supports: step-by-step sequences (linear, **Li**), exploring multiple approaches simultaneously (multipath, **Mp**), or responding adaptively without predetermined plans (reactive, **Re**)
   - *Reasoning (Reas) approaches the framework enables:* agents that think through problems step by step before acting (chain of thought, **CoT**), agents that jump straight to using tools (tool first, **Tf**), or agents that combine both approaches (hybrid, **Hy**)
   - *Decision-making (**Dec**) capabilities that the framework supports:* agents following fixed rules (deterministic, **D**), agents using AI models to make choices [large language model (LLM) driven, **L**], or agents that involve humans in key decisions (human in the loop, **H**).
3. *Architectural patterns:* At a structural level, frameworks leverage different philosophies for organizing agents. This determines whether the framework supports single-agent orchestration, such as those that can support workflow management, or *multiagent collaboration* with peer-to-peer coordination, or whether it implements *hierarchical structures* featuring supervisor-worker relationships. Control mechanisms vary from *centralized coordination* through a single orchestrator to *decentralized autonomy*, where agents self-organize to *hybrid models* that combine both approaches contextually. The architectural pattern dimension is characterized by the following:
   - *Structure:* the structural arrangements supported by the framework: one main agent handling all tasks (single agent, **Sa**), multiple specialized agents working together (multiagent, **Ma**), or organized teams with managers and workers (hierarchical, **Hie**)

- *Coordination mechanism supported by the framework:* agents working together through a central controller (centralized, **Cen**), agents communicating directly with each other (decentralized, **Dec**), or combining both approaches (hybrid, **Hy**)
- *Coupling between the components of the framework:* if everything is built as one unit (monolithic, **Mo**), separate interchangeable pieces (modular, **Mod**), or extensible through add-on components (plugin based, **Pl**).

4. *State and memory management:* Memory represents a critical differentiator in an agentic system's ability to maintain context and continuity. Frameworks vary in their persistence models; some support *stateless* or *session only*, while others support *file-based*, *database-backed*, or *vector-store* memory. State scope may be *local* (per agent), *shared* (across agents), or *global* (system wide). To determine this, the following characteristics are used:
   - *Memory:* the persistence options supported by the framework: temporary storage during conversations only (session only, **Se**), saving to local files (file based, **F**), storing in databases (database), or using specialized vector storage for semantic search (vector store, **V**)
   - *Scope:* the memory-sharing mechanism enabled by the framework: each agent keeping its own private memory (local, **Lo**), agents sharing memory with each other (shared, **Sh**), or all agents accessing system-wide memory (global, **G**)
   - *Context:* the context retention mechanism supported by the framework: agents with no memory between interactions (stateless, **st**), agents remembering recent conversation (short-term context, **ShT**), or agents maintaining persistent knowledge over time (long-term memory, **LT**).
   - *Number of stars in GitHub:* Higher stars suggest broader community interest and adoption.
   - *Number of forks:* More forks often indicate a project that developers want to build upon or customize.
   - *Number of contributors:* More contributors potentially indicate better long-term maintenance and diverse expertise.

> We selected 10 frameworks that represent the current landscape's diversity in architectural approaches, maturity levels, and target use cases.

The Selected Agentic AI Frameworks

We selected 10 frameworks that represent the current landscape's diversity in architectural approaches, maturity levels, and target use cases. The final selection includes both established platforms (AutoGPT, Semantic Kernel, and LangGraph) and emerging frameworks (Agno), spanning different approaches from multiagent collaboration systems to single-agent workflow orchestrators. Table 1 compares the frameworks against the selected five key dimensions, while Table 2 reports information on the supported LLM and on the community activities (the number of stars in GitHub, number of forks, and number of contributors).

In addition to the dimensions, we also characterize the agentic AI frameworks based on the community and the ecosystem support they provide. We evaluate this using the following characteristics:

- *LLM support:* The types of AI models that the framework enables are: proprietary models only (closed source), open source models only, or both. Mixed support offers more flexibility and reduces vendor lock-in risks.
- *Deployment:* This characterizes where the framework allows users to run agents: on premises, cloud services only, or both. Multiple options provide flexibility for different security, cost, and control requirements.
- *Language:* This refers to the programming language(s) supported by the framework. Some may support only one language, while some support multiple languages, thereby enabling wider integration.

Summary of Agentic AI Frameworks

This section summarizes the characteristics of the different agentic AI frameworks.

Table 1: Comparison of agentic AI frameworks across features.

Framework	Goal handling and interaction			Planning and reasoning			Architectural patterns			State and memory management		
	Goal processing	Input modality	Interact. patterns	Planning	Reasoning	Decision making	Structure	Coordination	Coupling	Memory	Scope	Context
AutoGPT	P	M	C, A	Mp	CoT, Tf	L	Ma	Dec	Mod	V	Sh	LT
Semantic Kernel	Pa, P	M	C	Hy	Tf, CoT	L, H	Sa	Cen	Pl, Mod	DB, V	Lo, Sh	ShT
LangGraph	P	M	C, A	Mp	Hy	L, H	Ma	Dec	Mod	V	Sh	LT
Agno	P	M	C	Mp	Hy	L	Ma	Dec	Mod	V	Sh	LT
MetaGPT	P	T	C	Mp	Hy	L	Hie	Hy	Mod	V	Sh, G	LT
CrewAI	P	M	C	Mp	Hy	L, H	Ma	Dec	Mod	V	Sh	LT
AgentGPT	Pa	T	C	Re	Tf	L	Sa	Cen	Mo	Se	Lo	St
SmolAgents	Pa	M	C	Li	Tf	L	Sa	Cen	Mod	Se	Lo	ShT
AutoGen	P	M	C, A	Mp	Hy	L, H	Ma	Hy	Mod, Pl	V	Sh, G	LT
Google ADK	P	M	C, A	Mp	Hy	L	Hie	Hy	Mod	V	Sh, G	LT

Interact.: interaction. Goal processing: P: proactive; Pa: passive. Input: T: text only; M: multimodal. Interaction: S, single shot; C: conversational; A: autonomous. Planning: Li, linear; Mp, multipath; Re: reactive. Reasoning: CoT: chain of thought; Tf: tool first; Hy, hybrid. Decision: D: deterministic; L: LLM driven; H: human in the loop. Structure: Sa: single agent; Ma: multiagent; Hie: hierarchical. Coordination: Cen: centralized; Dec: decentralized; Hy: hybrid. Coupling: Mo: monolithic; Mod: modular; Pl: plugin based. Memory: Se: session only; F: file based; DB: database; V: vector store. Scope: Lo: local; Sh: shared; G: global. Context: St: stateless; ShT: short term; LT: long term.

Goal Handling and Interaction

Most agentic AI frameworks support proactive goal processing, indicating a shift toward autonomous agent behavior. However, simpler frameworks, like AgentGPT and SmolAgents, still require passive and explicit instructions. From an input modality perspective, most frameworks support multimodal inputs, but some of the frameworks, like AgentGPT and MetaGPT, are primarily text driven. Conversational interaction is nearly universal, with advanced frameworks (Google ADK, AutoGen, and AutoGPT) adding autonomous loop capabilities, allowing agents to continuously operate and adapt without repeated user intervention. Semantic Kernel supports plan execution without user prompting—but not continuous autonomous loops.

Planning and Reasoning

Multipath planning has become the standard approach, with only simpler frameworks (AgentGPT and SmolAgents) using linear or reactive planning. There is a clear emphasis on multipath or hybrid planning (AutoGPT, MetaGPT, CrewAI, AutoGen, LangGraph, and Agno), indicating that agentic AI frameworks are converging on graph- or workflow-style orchestration rather than simple sequential steps. Reasoning often blends chain of thought with tool use (for example, AutoGPT and MetaGPT), while lighter frameworks (SmolAgents and AgentGPT) remain tool first. From a decision-making perspective, LLM-driven decision dominates across most of the frameworks. However, frameworks like Semantic Kernel, CrewAI, LangGraph, and AutoGen also provide human-in-the-loop options, which might be highly valuable for better reliability.

Architectural Patterns

Multiagent architectures are now predominant, reflecting the evolution from single-agent systems toward collaborative approaches. Most frameworks support multiagent collaboration (AutoGPT,

LangGraph, CrewAI, Agno, and AutoGen), with coordination shifting toward decentralized or hybrid approaches, while MetaGPT extends this further into a hierarchical multiagent structure. In contrast, lighter frameworks, like AgentGPT, SmolAgents, and Semantic Kernel, still operate primarily in a single-agent mode. In general, it can be observed that decentralized coordination is favored over centralized control, enabling more scalable and flexible agent interactions. Coupling is trending toward modular or plugin based (for example, Semantic Kernel, LangGraph, AutoGen, CrewAI, Google ADK, and SmolAgents), which enhances extensibility (integration of external plugins, databases, etc.), while AgentGPT remains more monolithic in nature.

State Management

Most of the agentic AI frameworks support vector storage for the persistence of long-term/shared memory (AutoGPT, LangGraph, Agno, MetaGPT, and CrewAI). Shared memory scope across agents is the dominant pattern, enabling effective collaboration, while global scope is reserved for hierarchical systems. Lightweight frameworks like AgentGPT and SmolAgents are session only or short term, making them easier to deploy, but they might be less suited to capture/maintain context, especially in long workflows. Among the frameworks, only MetaGPT and Google ADK support a global state, which is essentially required for handling system-wide orchestration. For LangGraph specifically, while vector stores are the most common persistence path, the framework also allows developers to plug in databases or alternative back ends for state management.

Community and Ecosystem

From a community perspective, AutoGPT dominates with 179,000 (179k) stars and hundreds of contributors, but agentic AI frameworks like MetaGPT (58.7k), AutoGen (50.2k), CrewAI (38.6k), and Agno (33.9k) also demonstrate wider adoption. Relatively new agentic AI frameworks like Google ADK (13.3k) and LangGraph (19.2k) are getting very high popularity, potentially indicating adoption. Most frameworks support creating agents that leverage proprietary/open source LLMs, while SmolAgents and Agno lean more toward open source LLMs. Most frameworks support cloud and on-premise/hybrid deployments; AgentGPT is cloud only, while Google ADK is hybrid (optimized for Google Cloud). Programming language ecosystems are also diverging; the majority are Python first, which simplifies integration for AI developers. Semantic Kernel is unique in providing multilanguage support (Python, C#, and Java). LangGraph supports both Python and JavaScript, bridging back-end and front-end ecosystems. Meanwhile, AgentGPT is built in TypeScript, reflecting its web application focus.

Beyond the Core Dimensions

Beyond the five analytical dimensions, several cross-cutting concerns shape the practical adoption of agentic AI frameworks. Some of these include observability support, security, interoperability, etc. Observability support varies widely; Semantic Kernel, Google ADK, and AutoGen provide strong tracing and telemetry, LangGraph offers partial support via LangSmith, while CrewAI and MetaGPT are limited to task logs. SmolAgents includes basic telemetry, but lighter

Table 2: Community adoption and ecosystem support of agentic AI frameworks.

Framework	Stars	Forks	Contributors	LLM support	Deployment	Language
AutoGPT	179k	46k	782	Mix	C, O	Python
Microsoft Semantic Kernel	26.3k	4.3k	412	Mix	C, O	Python, C#, Java
LangGraph	19.2k	3.4k	268	Mix	C, O	Python, JS
Agno	33.9k	4.3k	306	OS	C, O	Python
MetaGPT	58.7k	7.1k	113	Mix	C, O	Python
CrewAI	38.6k	5.1k	274	Mix	C, O	Python
AgentGPT	35k	9.5k	73	PP	C	JS/TS
SmolAgents	23.1k	2k	185	OS	C, O	Python
AutoGen	50.2k	7.7k	558	Mix	C, O	Python
Google ADK (Python)	13.3k	2k	138	Mix	C, O	Python

k: thousand; LLM support: OS: open source models; PP: proprietary only; Mix: both. Deployment: C: cloud; O: on premise.

Most agentic AI frameworks are still immature, with limited memory, basic planning, and uneven ecosystem support. Practitioners should carefully evaluate stability and integration options as scalability and reliability remain open challenges.

ABOUT THE AUTHORS



KARTHIK VAIDHYANATHAN is an assistant professor at the Software Engineering Research Center, IIIT-Hyderabad, Hyderabad 500032, India. Contact the author at karthik.vaidhyanathan@iiit.ac.in



DAVIDE TAIBI is the chair of software architecture and a center leader at the University of Southern Denmark, 7100 Vejle, Denmark. Contact the author at Davide.Taibi@oulu.fi.

frameworks like Agno leave observability largely to developers. Security remains largely underdeveloped, with only limited safeguards, such as SmolAgent's sandboxed execution or Google ADK's policy hooks. At the same time, interoperability is emerging as a key differentiator, with frameworks adopting standards like Model Context Protocol (MCP) (AutoGen, LangGraph, Google ADK, and CrewAI) and A2A protocols (CrewAI, LangGraph, and Google ADK) to enable richer tool integration and cross-agent collaboration.

This article presents an overview of current agentic AI frameworks, analyzing them along key dimensions such as goal handling and interactions, planning and reasoning, architectural patterns, state and memory management, and community and ecosystem. Our findings highlight that, while the frameworks offer diverse approaches, ranging from single-agent orchestrators to multiagent collaboration systems, most remain at an early stage of maturity. Memory management is often limited to short-term or vector-store implementations, and planning mechanisms frequently rely on simple multipath or reactive strategies. Ecosystem support also varies widely, with many frameworks still lacking robust documentation, scalable deployment options, and advanced error-handling mechanisms.

Future work should focus on improving persistence and memory models, supporting hybrid reasoning strategies, strengthening error recovery and scalability, supporting observability and security, and expanding tool ecosystems. Addressing these shortcomings will be key to making agentic AI frameworks more reliable and effective for real-world software engineering applications.

References

1. Z. Xi et al., "The rise and potential of large language model based agents: A survey," *Sci. China Inf. Sci.*, vol. 68, no. 2, 2025, Art. no. 121101, doi: 10.1007/s11432-024-4222-0.
2. Y. Liu et al., "Agent design pattern catalogue: A collection of architectural patterns for foundation model based agents," *J. Syst. Softw.*, vol. 220, Feb. 2025, Art. no. 112278, doi: 10.1016/j.jss.2024.112278.
3. D. B. Acharya, K. Kuppan, and B. Divya, "Agentic AI: Autonomous intelligence for complex goals—A comprehensive survey," *IEEE Access*, vol. 13, pp. 18,912–18,936, 2025, doi: 10.1109/ACCESS.2025.3532853.